

# Vitals: Clinical Data Integration with Measured Duplicate Resolution and Explainable Risk Flags

Paulo Henrique · July 2026 · [github.com/pharaujo-git/vitals](https://github.com/pharaujo-git/vitals)

## Abstract

Vitals is a health information system built around one research question: *how cleanly can heterogeneous clinical sources be consolidated into a single FHIR-compatible store, and at what cost in unresolved duplicate identities and silent data loss?* The system implements the full clinical workflow — patient records, scheduling, observations, messaging, consent, and auditing — and ingests three source formats (CSV extracts, HL7-style messages, FHIR R4 bundles) through one validated mapping pipeline. Two reproducible experiments back the design. The first measures the integration pipeline end-to-end on synthetically corrupted feeds: 100% of malformed records surface as reported issues, and an iteration on the duplicate matcher — motivated directly by the first measurement — lifted recall from 0.68 to 1.00 at precision 0.99. The second benchmarks the product's explainable rule-based risk score against a fitted logistic regression on a cohort with a known generative process, quantifying the accuracy/explainability trade (AUC 0.78 vs 0.81). All data is synthetic; the system, tests, and experiments are fully reproducible.

## 1. Motivation

Clinical data about one patient routinely lives in several systems that disagree on identifiers, formats, and spelling. Consolidation faces three recurring failures: records that silently drop when they do not fit the target schema, the same person existing twice because sources disagree on a name or a date of birth, and decision-support signals that clinicians cannot interrogate. Vitals treats all three as measurable engineering problems rather than aspirations: every mapping failure must be reported, duplicate-resolution quality must be scored against ground truth, and every risk flag must carry its own explanation.

## 2. System overview

Vitals is a FastAPI + PostgreSQL backend behind a React 19 front end, organized in strict layers (HTTP controllers → services → repositories → ORM), with Alembic migrations per feature. The clinical core covers patient records with problem lists, medications, allergies, imaging/document attachments and vitals trends; encounters whose observations are validated against a typed catalog with physiologic ranges and LOINC codes; scheduling with a per-clinician week calendar, overlap checks and a next-free-slot search; internal messaging with threads and attachments; and a population dashboard. Cross-cutting mechanics include four-role RBAC, per-record consent rules whose denials are themselves audited, an insert-only audit trail, refresh-token rotation with reuse detection, server-sent-event change signals in place of polling, and server-side pagination on every growable list.

Ingestion is the heart. CSV rows, pipe-delimited HL7-style segments, and FHIR R4 resources (validated against the FHIR schema via `fhir.resources`) all map through the same rules: patients match by MRN and are never overwritten by an import, observations pass the same catalog validation as manual entry, and every record that fails mapping becomes a persisted, reviewable issue attached to its import batch — with its raw source line. Exports speak FHIR R4: Patient, Observation, Condition, MedicationStatement and AllergyIntolerance resources in one bundle.

Quality gates: 66 backend tests over the service layer, 21 Playwright end-to-end tests, and a CI pipeline (tests, typecheck, build, e2e) on every push. A seed script generates a 60-patient synthetic population.

### 3. Experiment 1 — integration quality and the duplicate-matching iteration

**Method.** A generator produces 200 synthetic people; 120 of them also arrive through a second "source system" under different MRNs. Half of that second feed is clean; 15% carries an inner-character name typo, 10% a first-letter typo, 25% a  $\pm 1$ -day date-of-birth shift; five malformed rows (unparseable dates, unknown codes, out-of-range values, missing identifiers) are injected. Both feeds run through the production ingestion code on a disposable database, the production duplicate scanner runs, and flags are scored against the known same-person pairs.

**Error reporting.** All 5 injected malformed rows (100%) were reported as batch issues with their raw source data; the clean feed imported 200/200.

**Duplicate detection, two iterations.** The baseline matcher (exact name+DOB, plus first-initial+surname+DOB) scored precision 0.99 / recall 0.68 — and the per-corruption breakdown showed exactly why: it caught 0/17 first-letter typos and 0/22 date shifts. The second iteration added two explainable tiers targeting those measured failures: bounded Damerau–Levenshtein name matching (distance  $\leq 2$  per component, short names exact, blocked by DOB) and a  $\pm 31$ -day DOB window on identical names. Scored on identical generated data:

| Matcher                                | True pairs | False pos. | Precision | Recall | F1   |
|----------------------------------------|------------|------------|-----------|--------|------|
| Baseline (exact tiers)                 | 81 / 120   | 1          | 0.99      | 0.68   | 0.80 |
| Enhanced (+ edit distance, DOB window) | 120 / 120  | 1          | 0.99      | 1.00   | 1.00 |

The remaining false positive is a coincidental name+DOB collision — the reason flags route to human merge/dismiss review rather than auto-merging. The harness stays in place to price future candidates (phonetic codes, nickname tables, household blocking) the same way.

### 4. Experiment 2 — explainable rules vs. logistic regression

**Method.** A synthetic cohort of 4,000 patients is generated with a *known* latent deterioration risk built from the same physiology the product's rule engine watches (blood pressure, glucose, HbA1c, BMI, SpO<sub>2</sub>, age), plus noise for unmeasured factors (15% positive base rate; 35% held-out test set).

The production rules — each threshold crossing adds points and a human-readable reason — are compared with a scikit-learn logistic regression on eight features.

| Model                                  | AUC   | Precision | Recall | F1   |
|----------------------------------------|-------|-----------|--------|------|
| Rule-based score (threshold $\geq 3$ ) | 0.781 | 0.27      | 0.81   | 0.41 |
| Logistic regression                    | 0.814 | 0.62      | 0.25   | 0.36 |

The fitted model recovers the generative structure (its coefficient ordering mirrors the latent weights) and buys  $\sim 3$  AUC points — the price of the rules' step thresholds. What the rules buy back is the product property this project optimizes for: every flag in the dashboard lists the exact thresholds crossed with measured values. The harness makes the next step concrete: evaluate richer *explainable* models (scorecards, shallow trees) against both baselines.

## 5. Limitations and future work

The corruption model is synthetic and favorable — real-world name variation (nicknames, transliteration, hyphenation) is harsher than character-level typos, and the small name pool overstates collision risk while understating variation. The HL7 dialect is simplified (no MLLP transport or ACKs), the risk rules are demonstration-grade rather than clinically validated, and the system is single-tenant with no real-data compliance posture (it is deliberately a synthetic-data research vehicle, not a medical product). Planned iterations: phonetic and nickname-aware matching scored by the same harness; probabilistic (Fellegi–Sunter-style) record linkage as a third matcher tier; explainable-model comparisons in the risk harness; and a real HL7v2 interface boundary.

## 6. Conclusion

Vitals shows a complete, working clinical-integration loop: heterogeneous sources in, one consolidated record out, with data loss made visible, identity resolution measured against ground truth and demonstrably improved by one research iteration, and decision support that explains itself. The codebase, seed data, experiments and this report regenerate from a single repository.